

UNITY 6

Unit 2 - Basic Gameplay

Guía práctica 3D paso a paso

Player Positioning · Food Flight · Random Animal Stampede · Collision Decisions

Objetivo

Construir el Prototype 2 de Unity Learn: mover al Player lateralmente, lanzar Food, generar Animals de forma aleatoria y automática, y resolver Collision mediante Collider, Trigger y Rigidbody.

Cómo utilizar esta guía

Cada bloque indica qué hacer, qué concepto aparece por primera vez y qué debe observarse antes de continuar.

- Trabaje con un proyecto 3D en Unity 6.0 o una versión Unity 6 compatible con el curso.
- Respete los nombres sugeridos para Scripts, GameObjects y Prefabs; esto facilita comparar su proyecto con la guía.
- Guarde los Scripts antes de volver al Unity Editor y espere a que termine la compilación.
- Pruebe cada bloque en Play Mode antes de avanzar al siguiente.

Contenido

Sección	Qué se desarrolla
1. Unit 2 - Introduction	Propósito y resultado final del prototipo
2. Lesson 2.1 - Player Positioning	Pasos 1-8: proyecto, Input System, movimiento y límites
3. Lesson 2.2 - Food Flight	Pasos 9-15: projectile, Prefabs, disparo y destrucción fuera de límites
4. Lesson 2.3 - Random Animal Stampede	Pasos 16-21: SpawnManager, Array, Random.Range y cámara Orthographic
5. Lesson 2.4 - Collision Decisions	Pasos 22-25: método de spawn, InvokeRepeating, Collision y Game Over
6. Scripts finales	Código consolidado para comparar al terminar
7. Lista de comprobación	Revisión funcional completa del Prototipo 2

1. Unit 2 - Introduction

En esta unidad se construye un juego 3D con vista superior. El Player se desplaza de izquierda a derecha y lanza Food hacia Animals que avanzan por el escenario.

- Mover un Player mediante el Input System.
- Limitar el movimiento con condiciones if.
- Crear y reutilizar Prefabs.
- Generar objetos con Instantiate.
- Seleccionar elementos de un Array con valores aleatorios.
- Detectar contactos mediante Collider y Trigger.
- Destruir GameObjects y mostrar mensajes en Console.

Nota: Unity Learn proporciona un paquete de archivos iniciales para Prototype 2. Se descarga desde Resources > Prototype 2 - Starter Files dentro de Lesson 2.1.

2. Lesson 2.1 - Player Positioning

Objetivo de la lección: preparar Prototype 2, colocar los elementos 3D principales y programar el movimiento horizontal del Player sin permitir que abandone el área de juego.

Paso 1/25. Crear el proyecto e importar Prototype 2

Se parte de un proyecto limpio y se incorporan los Assets suministrados por Unity.

- Abra Unity Hub y cree un proyecto nuevo de tipo 3D con el nombre Prototype 2.
- En Unity Learn, abra Lesson 2.1 y descargue Prototype 2 - Starter Files desde la sección Resources.
- Descomprima el archivo descargado e importe el archivo .unitypackage en el proyecto.
- En la ventana Project, abra la escena Prototype 2.
- Elimine SampleScene si continúa dentro del proyecto y guarde la escena de trabajo.
- Si después de importar aparecen warnings antiguos en Console, use Clear y continúe siempre que no existan errores de compilación.

Conceptos nuevos

Asset: Recurso utilizado por el proyecto: modelo 3D, Material, Script, sonido, textura, escena, entre otros.

Project window: Ventana donde se organizan los archivos y Assets guardados en el proyecto.

Scene: Espacio de trabajo donde se colocan y configuran los GameObjects que forman un nivel.

Console: Ventana que muestra mensajes, warnings y errores generados por Unity o por los Scripts.

Comprobación: La escena Prototype 2 abre correctamente y puede observar el Ground y los recursos incluidos en el paquete.

Paso 2/25. Agregar Player, Animals y Food

Ahora se construye una escena mínima con un personaje, tres animales y un objeto de comida.

- Si desea cambiar la apariencia del Ground, arrastre un Material desde Course Library > Materials sobre el Ground.
- Arrastre a la Hierarchy un modelo Human, tres modelos Animal y un objeto Food.
- Cambie el nombre del personaje Human a Player.
- Distribuya temporalmente los Animals sobre el terreno para poder verlos desde la cámara.
- Ajuste la Scale del Food en X, Y y Z si resulta demasiado pequeño para verse desde arriba.

Conceptos nuevos

Hierarchy: Ventana que muestra los GameObjects que existen actualmente en la Scene.

GameObject: Objeto básico de Unity. Puede representar un personaje, cámara, luz, alimento, animal u objeto vacío.

Material: Recurso que controla la apariencia visual de una superficie, por ejemplo, color y textura.

Transform: Componente presente en todo GameObject que almacena Position, Rotation y Scale.

Scale: Tamaño del GameObject en los ejes X, Y y Z.

Comprobación: Player, tres Animals y Food aparecen en la Scene y pueden seleccionarse desde la Hierarchy.

Paso 3/25. Configurar el Input System

Unity Learn utiliza el Input System actual para leer las teclas de movimiento y disparo.

- Abra Edit > Project Settings.
- Seleccione Player en el panel izquierdo.
- Busque Configuration > Active Input Handling.
- Seleccione Both.
- Pulse Apply. Unity Editor se reiniciará para aplicar el cambio.

Conceptos nuevos

Input System: Sistema de Unity para leer teclado, mouse, gamepad y otros dispositivos de entrada.

Active Input Handling: Configuración que define qué sistema de entrada utiliza el proyecto. Both permite usar el sistema clásico y el nuevo Input System.

Comprobación: Al volver a abrir el proyecto, Active Input Handling permanece configurado en Both.

Paso 4/25. Crear PlayerController y configurar moveAction

El movimiento se controlará desde un Script C# conectado al Player.

- Dentro de Assets cree una carpeta llamada Scripts.
- Dentro de Scripts cree un Script llamado PlayerController.
- Arrastre PlayerController sobre el GameObject Player o use Add Component.
- Abra PlayerController y agregue el namespace UnityEngine.InputSystem.
- Declare una variable pública de tipo InputAction llamada moveAction.
- Guarde el Script y vuelva a Unity.
- Seleccione Player. En el componente PlayerController, despliegue Move Action.
- Pulse + Add Binding > Add Up/Down/Left/Right Composite.
- Asigne Left = keyboard/leftArrow y Right = keyboard/rightArrow. Para esta unidad no es obligatorio utilizar Up y Down.

PlayerController.cs - configuración inicial

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerController : MonoBehaviour
{
    public InputAction moveAction;

    void Start()
    {
        moveAction.Enable();
    }

    void Update()
    {
    }
}
```

Conceptos nuevos

Script: Archivo C# que define comportamiento para uno o varios GameObjects.

InputAction: Acción de entrada configurable, por ejemplo Move, Fire o Jump.

Binding: Relación entre una InputAction y un control físico, como una tecla.

Composite: Binding formado por varios controles que se combinan para producir un valor de movimiento.

Comprobación: En el Inspector del Player se observa Move Action con bindings para Left Arrow y Right Arrow.

Paso 5/25. Leer el movimiento como Vector2

Antes de mover el Player se comprobará qué valores produce el teclado.

- Declare public Vector2 moveInput; dentro de PlayerController.
- En Start(), confirme que moveAction.Enable(); está ejecutándose.
- En Update(), lea la acción con moveInput = moveAction.ReadValue<Vector2>();.
- Guarde y entre en Play Mode.
- Seleccione Player en la Hierarchy y observe Move Input en el Inspector.
- Pulse Left Arrow: el valor X debe llegar a -1. Pulse Right Arrow: el valor X debe llegar a 1.

Fragmento que lee la entrada

```
public Vector2 moveInput;

void Update()
{
    moveInput = moveAction.ReadValue<Vector2>();
}
```

Conceptos nuevos

Vector2: Estructura con dos valores, X y Y. En este caso X representa movimiento horizontal.

ReadValue<T>(): Método que obtiene el valor actual producido por una InputAction.

Play Mode: Modo de ejecución dentro del Editor usado para probar la Scene.

Comprobación: Move Input X cambia entre -1, 0 y 1 según la tecla horizontal presionada.

Paso 6/25. Mover el Player horizontalmente

El valor X leído del teclado se utilizará para desplazar el Player.

- Declare public float speed = 10.0f;.
- Después de leer moveInput, agregue el desplazamiento horizontal con transform.Translate.
- Guarde, entre en Play Mode y pruebe Left Arrow y Right Arrow.
- Si el movimiento es muy lento o rápido, ajuste Speed desde el Inspector.

Movimiento horizontal dentro de Update()

```
moveInput = moveAction.ReadValue<Vector2>();
transform.Translate(Vector3.right * moveInput.x * Time.deltaTime * speed);
```

Conceptos nuevos

Translate: Método de Transform que desplaza un GameObject en una dirección determinada.

Vector3.right: Dirección positiva del eje X: (1, 0, 0).

Time.deltaTime: Tiempo transcurrido desde el frame anterior; permite que la velocidad sea estable entre equipos.

speed: Variable utilizada para controlar la rapidez del desplazamiento.

Comprobación: El Player se mueve de izquierda a derecha de forma suave y la velocidad puede modificarse desde el Inspector.

Paso 7/25. Mantener el Player dentro del área de juego

El Player no debe poder salir por los laterales. Para ello se comprobará su posición X antes del movimiento.

- Declare public float xRange = 10.0f;.
- En Update(), compruebe si transform.position.x es menor que -xRange.
- Si se supera el límite izquierdo, reasigne la Position usando -xRange como X.
- Repita la comprobación para el lado derecho con transform.position.x > xRange.
- Mantenga los valores actuales de Y y Z al reconstruir el Vector3.

Límites laterales del Player

```
if (transform.position.x < -xRange)
{
    transform.position = new Vector3(-xRange, transform.position.y, transform.position.z);
}

if (transform.position.x > xRange)
{
    transform.position = new Vector3(xRange, transform.position.y, transform.position.z);
}
```

Conceptos nuevos

if: Estructura condicional que ejecuta un bloque solamente cuando una condición es verdadera.

< y >: Operadores de comparación: menor que y mayor que.

position: Propiedad de Transform que contiene la ubicación X, Y y Z del GameObject.

xRange: Valor máximo permitido para el desplazamiento horizontal del Player.

Comprobación: Al mantener presionada una flecha, el Player llega al borde definido y deja de avanzar fuera del área.

Paso 8/25. Ordenar el Script y comprobar Lesson 2.1

Se integra el código de movimiento con las condiciones de límite y se realiza una prueba completa.

PlayerController.cs al terminar Lesson 2.1

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerController : MonoBehaviour
{
    public InputAction moveAction;
    public Vector2 moveInput;
    public float speed = 10.0f;
    public float xRange = 10.0f;

    void Start()
    {
        moveAction.Enable();
    }

    void Update()
    {
        // Mantiene al Player dentro de los límites laterales.
        if (transform.position.x < -xRange)
        {
            transform.position = new Vector3(-xRange, transform.position.y, transform.position.z);
        }
        if (transform.position.x > xRange)
        {
            transform.position = new Vector3(xRange, transform.position.y, transform.position.z);
        }

        moveInput = moveAction.ReadValue<Vector2>();
        transform.Translate(Vector3.right * moveInput.x * Time.deltaTime * speed);
    }
}
```

Comprobación: Lesson 2.1 terminada: el Player responde a Left/Right Arrow y nunca abandona el área por los laterales.

3. Lesson 2.2 - Food Flight

Objetivo de la lección: convertir Food en un projectile reutilizable, lanzarlo con Spacebar y eliminar projectiles y Animals cuando abandonen los límites de la Scene.

Paso 9/25. Hacer que Food avance hacia delante

Food necesita movimiento propio para funcionar como projectile.

- Cree un Script llamado MoveForward dentro de Assets/Scripts.
- Añádalo al GameObject Food.
- Declare public float speed = 40.0f;.
- En Update(), desplace el objeto con Vector3.forward.
- Pruebe en Play Mode y ajuste Speed si lo necesita.

MoveForward.cs

```
using UnityEngine;

public class MoveForward : MonoBehaviour
{
    public float speed = 40.0f;

    void Update()
    {
        transform.Translate(Vector3.forward * Time.deltaTime * speed);
    }
}
```

Conceptos nuevos

Vector3.forward: Dirección positiva del eje Z: (0, 0, 1). En este prototipo corresponde al avance hacia la parte superior del campo.

Frame: Imagen individual del juego. Update() se ejecuta una vez por cada frame.

Comprobación: Al iniciar Play Mode, Food avanza continuamente sobre el eje Z.

Paso 10/25. Convertir Food en un Prefab

El projectile debe poder crearse muchas veces sin reconstruirse manualmente.

- Cree una carpeta Assets/Prefabs.
- Arrastre Food desde la Hierarchy hacia la carpeta Prefabs.
- Cuando Unity lo solicite, seleccione Original Prefab.
- En PlayerController declare public GameObject projectilePrefab;.
- Seleccione Player y arrastre el Food Prefab desde Project hasta Projectile Prefab en el Inspector.
- Puede eliminar la instancia temporal de Food de la Hierarchy cuando ya no la necesite.

Conceptos nuevos

Original Prefab: Prefab independiente creado a partir de un GameObject de la Scene.

GameObject variable: Referencia que permite a un Script guardar y utilizar otro GameObject o Prefab.

Comprobación: PlayerController muestra Projectile Prefab y contiene la referencia al Food Prefab.

Paso 11/25. Configurar fireAction para Spacebar

Se crea una segunda InputAction, esta vez para lanzar el projectile.

- En PlayerController declare public InputAction fireAction;.
- En Start(), agregue fireAction.Enable();.
- Guarde y vuelva al Inspector del Player.
- Despliegue Fire Action y pulse + Add Binding.
- Configure Path como keyboard/space. También puede usar Listen y presionar Spacebar.

Conceptos nuevos

fireAction: InputAction usada para detectar la orden de lanzar el projectile.

triggered: Propiedad que es true en el frame en el que una InputAction se ejecuta.

Comprobación: Fire Action queda enlazada con Spacebar y no aparecen errores en Console.

Paso 12/25. Detectar Spacebar y lanzar el projectile

Se integran en un mismo bloque la detección de Fire Action y la creación del Food Prefab.

- Dentro de Update(), cree una condición if (fireAction.triggered).

- Dentro del if, llame Instantiate.
- Use projectilePrefab como objeto a crear.
- Use transform.position del Player como punto de aparición.
- Use projectilePrefab.transform.rotation para conservar la orientación del Prefab.
- Pruebe presionando Spacebar varias veces.

Disparo del Food Prefab

```
if (fireAction.triggered)
{
    Instantiate(projectilePrefab, transform.position, projectilePrefab.transform.rotation);
}
```

Conceptos nuevos

Instantiate: Método que crea una nueva instancia de un GameObject o Prefab durante la ejecución.

rotation: Orientación de un Transform expresada internamente como Quaternion.

Comprobación: Cada pulsación de Spacebar crea un nuevo Food en la Position del Player y este avanza hacia delante.

Paso 13/25. Convertir los Animals en Prefabs

Los Animals también deben convertirse en Prefabs porque más adelante aparecerán mediante SpawnManager.

- Rote cada Animal 180 grados en el eje Y para que mire hacia el Player.
- Seleccione los Animals y añada el componente MoveForward.
- Ajuste Speed y pruebe que caminen hacia la parte inferior del campo.
- Arrastre cada Animal a Assets/Prefabs y seleccione Original Prefab.
- Pruebe arrastrando una instancia desde Project a la Scene durante Play Mode: debe moverse automáticamente.

Conceptos nuevos

Component: Pieza de funcionalidad añadida a un GameObject, por ejemplo Transform, Collider, Rigidbody o un Script.

Prefab instance: Copia de un Prefab colocada en una Scene; mantiene una relación con su Prefab original.

Comprobación: Los tres Animals existen como Prefabs y cada uno se mueve hacia el Player al aparecer en la Scene.

Paso 14/25. Destruir proyectiles fuera del límite superior

Los proyectiles que salen del campo deben eliminarse para no acumular GameObjects innecesarios.

- Cree un Script llamado DestroyOutOfBounds.
- Añádalo a una instancia de Food o edite directamente el Food Prefab.
- Declare private float topBound = 30;.
- En Update(), compruebe si transform.position.z > topBound.
- Si la condición es verdadera, ejecute Destroy(gameObject);.
- Si modificó una instancia de Prefab en la Scene, use Overrides > Apply All para guardar el Script en el Prefab.

Primera versión de DestroyOutOfBounds.cs

```
private float topBound = 30;

void Update()
{
    if (transform.position.z > topBound)
    {
        Destroy(gameObject);
    }
}
```

Conceptos nuevos

Destroy: Método que elimina un Object de Unity durante la ejecución.

gameObject: Referencia al GameObject al que está unido el Script actual.

private: Modificador que limita el acceso directo a una variable desde otras clases y, por defecto, no la muestra en Inspector.

Override: Cambio realizado en una instancia de Prefab que puede aplicarse al Prefab original.

Comprobación: Food desaparece al superar aproximadamente Z = 30.

Paso 15/25. Destruir Animals fuera del límite inferior

El mismo Script servirá para detectar objetos que abandonan el campo por la parte inferior.

- En DestroyOutOfBounds declare private float lowerBound = -10;.
- Después de la condición del topBound, agregue else if (transform.position.z < lowerBound).
- Dentro del nuevo bloque ejecute Destroy(gameObject);.
- Añada DestroyOutOfBounds a cada Animal Prefab.
- Pruebe que un Animal desaparezca al pasar la zona inferior del Player.

DestroyOutOfBounds.cs al terminar Lesson 2.2

```
using UnityEngine;

public class DestroyOutOfBounds : MonoBehaviour
{
    private float topBound = 30;
    private float lowerBound = -10;

    void Update()
    {
        if (transform.position.z > topBound)
        {
            Destroy(gameObject);
        }
        else if (transform.position.z < lowerBound)
        {
            Destroy(gameObject);
        }
    }
}
```

Conceptos nuevos

else if: Condición alternativa que solo se evalúa cuando el if anterior fue falso.

Bounds: Límites numéricos usados para decidir cuándo un objeto está fuera del área válida.

Comprobación: Lesson 2.2 terminada: Spacebar lanza Food; Food y Animals se eliminan al salir de los límites correspondientes.

4. Lesson 2.3 - Random Animal Stampede

Objetivo de la lección: crear un SpawnManager capaz de seleccionar un Animal al azar y colocarlo en una Position X aleatoria en la parte superior del campo. Al final se cambia la Camera a Orthographic.

Paso 16/25. Crear SpawnManager y el Array de Animals

La responsabilidad de generar Animals se separa del PlayerController.

- En la Hierarchy cree Create Empty y cambie el nombre a SpawnManager.
- Cree un Script llamado SpawnManager y añádalo al GameObject SpawnManager.
- Abra el Script y declare `public GameObject[] animalPrefabs;`
- Guarde y seleccione SpawnManager en la Hierarchy.
- En el Inspector cambie Size del Array al número de Animals utilizados, por ejemplo 3.
- Arrastre los Animal Prefabs desde Project a Element 0, Element 1 y Element 2. No use instancias de la Hierarchy.

Conceptos nuevos

Empty GameObject: GameObject sin representación visual que se utiliza para organizar o ejecutar lógica mediante Scripts.

SpawnManager: GameObject y Script dedicados a controlar cuándo, dónde y qué objeto aparece.

Array: Colección de elementos del mismo tipo almacenados mediante posiciones llamadas índices.

Index: Posición de un elemento dentro de un Array. El primer índice es 0.

Comprobación: SpawnManager muestra un Array con todos los Animal Prefabs asignados desde la ventana Project.

Paso 17/25. Configurar una spawnAction temporal

Unity Learn primero usa la tecla S para comprobar manualmente que el sistema de spawn funciona.

- Agregue `using UnityEngine.InputSystem;` en SpawnManager.cs.
- Declare `public InputAction spawnAction;`
- En `Start()`, ejecute `spawnAction.Enable();`
- Vuelva a Unity y despliegue Spawn Action en el Inspector.
- Añada un Binding con Path = keyboard/s.

Conceptos nuevos

spawnAction: InputAction temporal usada para solicitar manualmente la aparición de un Animal con la tecla S.

Comprobación: Al seleccionar SpawnManager se observa Spawn Action enlazada a la tecla S.

Paso 18/25. Generar un Animal específico con S

Antes de introducir aleatoriedad se valida que Instantiate funciona con un elemento concreto del Array.

- Declare temporalmente `public int animalIndex;`
- En `Update()`, cree `if (spawnAction.triggered)`.
- Dentro del if, use `Instantiate(animalPrefabs[animalIndex], ...)`.
- Utilice una Position en la parte superior de la Scene, por ejemplo Z = 20.
- Cambie Animal Index desde el Inspector para comprobar que puede generar diferentes Animals.

Prueba inicial del Array

```
if (spawnAction.triggered)
{
    Instantiate(animalPrefabs[animalIndex],
        new Vector3(0, 0, 20),
        animalPrefabs[animalIndex].transform.rotation);
}
```

Comprobación: Al presionar S aparece el Animal correspondiente al índice seleccionado.

Paso 19/25. Elegir un Animal al azar

El índice deja de escribirse manualmente y pasa a generarse con Random.Range.

- Elimine la variable global public int animalIndex;.
- Dentro del if, cree int animalIndex = Random.Range(0, animalPrefabs.Length);.
- Mantenga el mismo Instantiate, utilizando ahora el índice generado localmente.
- Presione S varias veces y observe cómo cambia el Animal seleccionado.

Selección aleatoria del índice

```
int animalIndex = Random.Range(0, animalPrefabs.Length);
```

Conceptos nuevos

Random.Range: Método que devuelve un valor aleatorio dentro de un rango. Con int, el límite máximo no se incluye.

Length: Propiedad que indica cuántos elementos contiene un Array.

Local variable: Variable declarada dentro de un método o bloque; solo existe y se usa en ese alcance.

Global/Class variable: Variable declarada en la clase; puede ser utilizada por varios métodos de esa clase.

Comprobación: Cada pulsación de S puede generar un Animal diferente sin cambiar valores en el Inspector.

Paso 20/25. Generar una Position X aleatoria

Los Animals no deben aparecer todos en una misma línea.

- Declare private float spawnRangeX = 20; y private float spawnPosZ = 20;.
- Dentro del if, cree Vector3 spawnPos.
- Use Random.Range(-spawnRangeX, spawnRangeX) para X.
- Mantenga Y = 0 y Z = spawnPosZ.
- Use spawnPos en Instantiate.
- Presione S repetidamente y observe diferentes posiciones laterales.

Position de spawn aleatoria

```
Vector3 spawnPos = new Vector3(
    Random.Range(-spawnRangeX, spawnRangeX),
    0,
    spawnPosZ);
```

Comprobación: Los Animals aparecen en posiciones X diferentes dentro de un rango controlado.

Paso 21/25. Cambiar la Camera a Orthographic

La unidad utiliza una vista superior y permite comparar Perspective con Orthographic.

- En Scene view alterne entre Perspective e Isometric para observar la diferencia visual.
- Seleccione Main Camera en la Hierarchy.
- En el componente Camera cambie Projection de Perspective a Orthographic.
- Ajuste Orthographic Size si necesita encuadrar mejor el campo de juego.
- Pruebe nuevamente el movimiento y el spawn.

Conceptos nuevos

Perspective: Proyección donde los objetos lejanos se ven más pequeños, similar a la visión humana.

Orthographic: Proyección sin reducción por distancia; resulta útil en vistas superiores y estilos 2D/3D planos.

Isometric: Vista visual sin punto de fuga pronunciado, usada para representar espacios 3D con sensación técnica o estratégica.

Projection: Propiedad de Camera que define cómo se transforma la escena 3D en la imagen mostrada al jugador.

SpawnManager.cs al terminar Lesson 2.3

```
using UnityEngine;
using UnityEngine.InputSystem;

public class SpawnManager : MonoBehaviour
{
    public GameObject[] animalPrefabs;
    public InputAction spawnAction;

    private float spawnRangeX = 20;
    private float spawnPosZ = 20;

    void Start()
    {
        spawnAction.Enable();
    }

    void Update()
    {
        if (spawnAction.triggered)
        {
            Vector3 spawnPos = new Vector3(
                Random.Range(-spawnRangeX, spawnRangeX), 0, spawnPosZ);

            int animalIndex = Random.Range(0, animalPrefabs.Length);

            Instantiate(animalPrefabs[animalIndex],
                spawnPos,
                animalPrefabs[animalIndex].transform.rotation);
        }
    }
}
```

Comprobación: Lesson 2.3 terminada: S genera un Animal aleatorio en una Position X aleatoria y la cámara puede utilizar Projection Orthographic.

5. Lesson 2.4 - Collision Decisions

Objetivo de la lección: reemplazar el spawn manual por un temporizador, preparar Collider y Rigidbody, detectar Trigger y destruir los objetos implicados. También se informa Game Over cuando un Animal supera al Player.

Paso 22/25. Extraer el spawn a un método propio

La lógica para generar Animals se mueve a un método para poder llamarla desde diferentes lugares.

- Debajo de Update(), cree void SpawnRandomAnimal() {}.
- Mueva al nuevo método el código que calcula animalIndex, spawnPos y ejecuta Instantiate.
- Durante esta comprobación puede dejar en Update() el if de spawnAction y llamar SpawnRandomAnimal(); dentro de él.
- Pruebe S para confirmar que el comportamiento no cambió.

Método SpawnRandomAnimal()

```
void SpawnRandomAnimal()
{
    int animalIndex = Random.Range(0, animalPrefabs.Length);
    Vector3 spawnPos = new Vector3(
        Random.Range(-spawnRangeX, spawnRangeX), 0, spawnPosZ);

    Instantiate(animalPrefabs[animalIndex],
        spawnPos,
        animalPrefabs[animalIndex].transform.rotation);
}
```

Method: Bloque de código con un nombre que realiza una tarea específica y puede llamarse desde otras partes de la clase.
void: Tipo de retorno que indica que un método no devuelve un valor.

Comprobación: La tecla S sigue generando Animals, pero el código de generación está concentrado en `SpawnRandomAnimal()`.

Paso 23/25. Generar Animals automáticamente con `InvokeRepeating`

Ya no será necesario presionar S: Unity llamará el método de spawn a intervalos regulares.

- Declare `private float startDelay = 2;`
- Declare `private float spawnInterval = 1.5f;`
- En `Start()`, agregue `InvokeRepeating("SpawnRandomAnimal", startDelay, spawnInterval);`.
- Elimine de `Update()` el `if` que esperaba la tecla S. Puede dejar `Update()` vacío.
- Entre en Play Mode y espere unos segundos.
- Ajuste `startDelay` y `spawnInterval` si desea observar con mayor comodidad el resultado.

Inicio del spawn automático

```
void Start()
{
    spawnAction.Enable();
    InvokeRepeating("SpawnRandomAnimal", startDelay, spawnInterval);
}
```

Conceptos nuevos

InvokeRepeating: Método de `MonoBehaviour` que llama repetidamente otro método después de un retraso inicial y con un intervalo definido.

startDelay: Tiempo de espera antes de ejecutar el primer spawn.

spawnInterval: Tiempo entre una aparición y la siguiente.

Nota: En la secuencia oficial, `spawnAction` permanece declarado aunque ya no se usa para generar Animals después de activar `InvokeRepeating`. Puede conservarlo para seguir fielmente el recorrido del tutorial.

Comprobación: Los Animals aparecen automáticamente sin presionar ninguna tecla.

Paso 24/25. Agregar Collider, Trigger y Rigidbody

Para detectar el contacto entre Food y Animals, ambos necesitan una forma de Collision válida.

- Abra un Animal Prefab en Prefab Mode haciendo doble clic sobre él en Project.
- Use Add Component > Box Collider.
- Active Edit Collider y ajuste el volumen para cubrir el modelo.
- Marque Is Trigger.
- Repita el proceso en los otros Animal Prefabs.
- Abra el Food Prefab y añada Box Collider; ajuste su tamaño y marque Is Trigger.
- Añada Rigidbody al Food Prefab.
- Desactive Use Gravity en Rigidbody.
- Verifique que los cambios quedan guardados en los Prefabs. Si trabajó sobre una instancia, use Overrides > Apply All.

Conceptos nuevos

Collider: Componente que define una forma física utilizada por Unity para detectar contactos y solapamientos.

Box Collider: Collider con forma de caja rectangular.

Trigger: Collider configurado para detectar que otro Collider entra en su volumen sin producir una respuesta física de choque.

Rigidbody: Componente que integra un GameObject con el sistema de física de Unity. Es necesario para muchos eventos de Collision/Trigger.

Use Gravity: Propiedad de Rigidbody que decide si la gravedad modifica el movimiento del objeto.

Prefab Mode: Modo de edición aislada de un Prefab; permite modificar directamente su plantilla.

Comprobación: Food tiene Rigidbody sin gravedad y tanto Food como Animals cuentan con Box Collider marcado como Is Trigger.

Paso 25/25. Destruir objetos por Trigger y registrar Game Over

Se completa la interacción: Food y Animal desaparecen al entrar en contacto y Console informa cuando un Animal atraviesa el límite inferior.

- Cree DetectCollisions.cs y añádalo a cada Animal Prefab.
- Dentro de la clase cree void OnTriggerEnter(Collider other).
- Dentro del método ejecute Destroy(gameObject); para eliminar el Animal.
- Ejecute Destroy(other.gameObject); para eliminar el Food que entró en el Trigger.
- Abra DestroyOutOfBounds.cs.
- En el bloque else if que comprueba lowerBound, agregue Debug.Log("Game Over!"); antes de Destroy(gameObject);.
- Pruebe el juego: lance Food hacia los Animals y deje que al menos uno llegue al límite inferior.

DetectCollisions.cs

```
using UnityEngine;

public class DetectCollisions : MonoBehaviour
{
    void OnTriggerEnter(Collider other)
    {
        Destroy(gameObject);
        Destroy(other.gameObject);
    }
}
```

Cambio final en DestroyOutOfBounds.cs

```
else if (transform.position.z < lowerBound)
{
    Debug.Log("Game Over!");
    Destroy(gameObject);
}
```

Conceptos nuevos

OnTriggerEnter: Método de evento que Unity ejecuta cuando un Collider entra en un Trigger compatible.

Collider other: Parámetro que representa el otro Collider implicado en el evento.

Debug.Log: Método que escribe un mensaje informativo en la ventana Console.

Game Over: Condición de pérdida del prototipo; en esta unidad se representa únicamente mediante un mensaje en Console.

Comprobación: Lesson 2.4 terminada: los Animals aparecen por intervalos; Food y Animal desaparecen al tocarse; si un Animal llega al borde inferior aparece Game Over! en Console.

6. Scripts finales para comparar

Estos Scripts reúnen el estado esperado al terminar las cuatro lecciones. Úselos para comparar nombres de variables, estructura y orden general. Si su proyecto funciona, no es necesario reemplazar un Script correcto solo porque el formato sea diferente.

PlayerController.cs

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerController : MonoBehaviour
{
    public InputAction moveAction;
    public Vector2 moveInput;
    public float speed = 10.0f;
    public float xRange = 10.0f;
    public GameObject projectilePrefab;
    public InputAction fireAction;

    void Start()
    {
        moveAction.Enable();
        fireAction.Enable();
    }

    void Update()
    {
        // Mantiene al Player dentro de los límites laterales.
        if (transform.position.x < -xRange)
        {
            transform.position = new Vector3(-xRange, transform.position.y, transform.position.z);
        }
        if (transform.position.x > xRange)
        {
            transform.position = new Vector3(xRange, transform.position.y, transform.position.z);
        }

        moveInput = moveAction.ReadValue<Vector2>();
        transform.Translate(Vector3.right * moveInput.x * Time.deltaTime * speed);

        if (fireAction.triggered)
        {
            Instantiate(projectilePrefab, transform.position, projectilePrefab.transform.rotation);
        }
    }
}
```

MoveForward.cs

```
using UnityEngine;

public class MoveForward : MonoBehaviour
{
    public float speed = 40.0f;

    void Update()
    {
        transform.Translate(Vector3.forward * Time.deltaTime * speed);
    }
}
```

DestroyOutOfBounds.cs

```

using UnityEngine;

public class DestroyOutOfBounds : MonoBehaviour
{
    private float topBound = 30;
    private float lowerBound = -10;

    void Update()
    {
        // Elimina objetos que dejan el área visible del juego.
        if (transform.position.z > topBound)
        {
            Destroy(gameObject);
        }
        else if (transform.position.z < lowerBound)
        {
            Debug.Log("Game Over!");
            Destroy(gameObject);
        }
    }
}

```

SpawnManager.cs

```

using UnityEngine;
using UnityEngine.InputSystem;

public class SpawnManager : MonoBehaviour
{
    public GameObject[] animalPrefabs;
    public InputAction spawnAction;

    private float spawnRangeX = 20;
    private float spawnPosZ = 20;
    private float startDelay = 2;
    private float spawnInterval = 1.5f;

    void Start()
    {
        spawnAction.Enable();
        InvokeRepeating("SpawnRandomAnimal", startDelay, spawnInterval);
    }

    void Update()
    {
    }

    void SpawnRandomAnimal()
    {
        int animalIndex = Random.Range(0, animalPrefabs.Length);
        Vector3 spawnPos = new Vector3(
            Random.Range(-spawnRangeX, spawnRangeX), 0, spawnPosZ);

        Instantiate(animalPrefabs[animalIndex],
            spawnPos,
            animalPrefabs[animalIndex].transform.rotation);
    }
}

```

DetectCollisions.cs

```

using UnityEngine;

public class DetectCollisions : MonoBehaviour
{
    void OnTriggerEnter(Collider other)
    {
        Destroy(gameObject);
        Destroy(other.gameObject);
    }
}

```

Complementos para Proyecto de Aula

Para superar la calificación de 2,0, el proyecto debe incluir funcionalidades que no estén resueltas directamente en el paso a paso de la guía. El equipo deberá investigar cómo implementarlas, adaptarlas a su juego y demostrar que funcionan correctamente.

Se deben implementar como mínimo 6 mejoras. Al menos 3 de ellas debe modificar la jugabilidad y no limitarse a un cambio visual.

- **Sistema de puntuación** — sumar puntos cada vez que se destruye un objetivo.
- **Vidas del jugador** — iniciar con 3 vidas y perder una cuando un enemigo llega al borde inferior.
- **Pantalla de Game Over** — mostrar un panel con mensaje, puntaje final y botón para reiniciar.
- **Aumento progresivo de dificultad** — reducir el spawnInterval o aumentar la velocidad de los enemigos con el tiempo.
- **Diferentes puntajes por enemigo** — que cada tipo de objetivo entregue una cantidad distinta de puntos.
- **Power-Up de disparo rápido** — recoger un objeto que permita disparar más rápido durante algunos segundos.
- **Power-Up de escudo** — permitir que el jugador soporte un error sin perder una vida.
- **Sonidos básicos** — agregar efectos de sonido al disparar, destruir enemigos y perder una vida.
- **Animación simple de destrucción** — mostrar una pequeña explosión antes de eliminar el enemigo.
- **Récord de puntuación** — guardar el puntaje más alto utilizando PlayerPrefs.
- **También se aceptarán otras mejoras propuestas por el equipo**, siempre que sean coherentes con el juego, tengan un nivel de complejidad razonable y puedan explicarse durante la sustentación.